

Ask the data — AI query in the Analysis tab

Research, design and build plan. 23 September 2026.

A crew member types a question the way they would say it out loud. They get back a written answer, the chart it came from, and the photo, clip or sail scan that shows it. This is the step from *dashboard* — where you must already know which table holds your answer — to *custom extraction*, where you only have to know the question.

It builds on [ai-analyst-roadmap-2026-07.md](#) and delivers its **Phase 4** (router + typed tools) on the surface people actually use, plus the media retrieval that roadmap did not cover.

1. Research — what best-in-class actually does in 2026

Nine sources, three that changed the design.

1.1 Nobody ships naked text-to-SQL any more

The single most consistent finding. Raw LLM text-to-SQL **solves about 21 % of real enterprise queries**; the tools that score well do it by constraining the model to a governed semantic layer rather than to a schema ([Querio benchmark round-up](#), [Holistics](#)). The category leaders — ThoughtSpot Spotter, Power BI, Looker — now route every AI query through curated metric definitions so that the answer matches what the official dashboard says. Where AI writes SQL against raw tables, "Revenue" means gross in one answer and net in the next, and two people asking the same question get different numbers.

The strongest primary source is [Beyond Text-to-SQL: An Agentic LLM System for Governed Enterprise Analytics APIs](#) (2026). Instead of generating SQL, their agent calls **governed API endpoints** — which already enforce tenant scoping, masking and business logic — and reaches **77.2 % end-to-end accuracy** (Gemini-2.5-Pro) / **71.7 %** on the cheaper model they actually shipped, with 96.7 % execution success.

Two findings from that paper are load-bearing here:

- **Deterministic date handling.** They deliberately take *all* date arithmetic away from the model and give it to plain functions, "to reduce hallucination". In SSA the equivalent trap is worse than dates: local-vs-UTC clocks (see [CLAUDE.md](#)). The model must never do arithmetic on a timestamp.
- **Structural validity is not correctness.** Their weakest model produced 44.4 % *executable* requests but only 17.4 % *correct* ones — "endpoint choice, target grounding and temporal filtering can all be subtly wrong even when the payload passes validation." A tool call that parses is not a tool call that answered the question. This is why the resolved filter has to be shown on screen, in words.

1.2 SSA already has the semantic layer — it just isn't exposed

This is the reason this feature is a fortnight's work and not a quarter's. The governed layer the research says to build already exists, tested against the KND report:

Layer	Where	What it gives the model
Grain	<code>PhaseStat</code> — one 30 s steady-state phase	The unit of performance, not "a row"
Metrics	<code>CHANNELS</code> in <code>phaseStats.ts</code>	40-odd named channels with units, decimals, plausibility caps
Dimensions	<code>GroupKey</code> = mode, tack, sailCombo, race, TWS/TWA/heel band	The only legal ways to slice
Aggregation	<code>groupPhases()</code>	Means of per-phase means, <code>n</code> per group, automatic band edges
Canonical reports	<code>REPORTS</code> in <code>reportTables.ts</code>	The KND tables, in KND's order

Layer	Where	What it gives the model
Season	<code>session_phase_stats</code> rows + <code>seasonCurves</code>	Cross-day comparison at matched wind

So the tools are thin typed wrappers over functions that already have tests. The model picks a tool and fills arguments; **TypeScript that KND validated does the maths.**

1.3 Constrain the chart too, not just the query

[Chat2VIS-lineage research](#) finds monolithic LLM chart generation handles simple charts and falls over on anything structural, while a hybrid that decomposes the task and applies **formal grammar constraints and schema validation** reaches **99.74 % execution success** (+38.4 points) — the conclusion being that formal constraints are the production foundation. [V-RECS](#) and the governed-API paper both pick the mark type with **rule-based logic** (line for temporal, bar for categorical).

→ **The model never emits a chart spec.** Each tool returns its own chart alongside its table, chosen by a rule from the shape of the result. A chart can then never disagree with the numbers under it, because it is drawn from them.

1.4 Show the interpretation, not the reasoning

ThoughtSpot's answer to explainability is not a chain of thought: it is [search tokens](#) — the question rendered back as the structured query it became, readable by a non-technical user, "so business users can verify results without reading code".

This is the highest-leverage UI idea in the whole round of research, and it fixes the §1.1 failure mode directly. A crew member cannot audit a tool call, but they can read [Upwind · 11 Sep · TWS 14–18 kn · by tack · 34 phases](#) and know instantly whether the machine understood them. It also makes a near-miss recoverable by tapping a chip, instead of re-typing a paragraph and hoping.

1.5 Ambiguity gets suggestions, not a guess

Snowflake Cortex Analyst does not answer an ambiguous question. It returns a [suggestion](#) content type — "*Your question is ambiguous, here are some alternatives*" — and the custom-instruction layer tells it when to reject and when to ask ([docs](#)). Its Verified Query Repository — a curated set of question → query pairs — is the same mechanism as the few-shot curation already planned from 👍 rows.

1.6 Sports analysis: the answer is a finding *plus the evidence*

Hudl's strength is explicitly that "video, scouting and performance data sit in one place"; Catapult Sportcode is built for tagging-plus-performance review; the newer NL layers let a coach say "*analyse all backhand loops*" and get **structured findings with timestamps, backed by video evidence clips** ([Folio3 round-up](#), [Pelayar](#)). 97 % of professional teams now run some AI analytics, in a \$6.3 bn market.

This is the part general BI copilots do *not* do, and it is exactly what SSA holds that Tableau does not: a photo carrying [analysis_data.inst](#) (TWS/TWA/BSP/heel at the shutter), a clip with [start_utc](#), a sail scan with [tws_kn / twa_deg](#) and stripe metrics, and the crew's own tags with notes. **An answer without the picture is a worse answer**, and it is the difference between "VMG was 3 % down in the puffs" and seeing the jib in that puff.

1.7 UX rules taken from Microsoft's HAX guidance

From [Microsoft's copilot UX guidance](#):

- **Suggestions to get going** — "long-form natural language typing is still not a habit for many". Never show an empty box.
- **Human in control**, in the wording: "*Summarize with copilot*", not "*Copilot, summarize*". → the button says **Ask**, not "AI".
- **Show inputs and outputs together**, keep a **history** — a later prompt is often worse than an earlier one.
- **Citations and direct quotes** encourage fact-checking; and showing references is not enough on its own.

- **Appropriate friction** at save/share/copy, with an AI notice on every output.
- **Withhold the output when necessary** — no answer beats a wrong one.
- **Granular feedback** during regular use.

And the governance warning worth designing against: Gartner expects **40 % of enterprises to demote or decommission autonomous agents by 2027** over governance gaps, the rolled-back ones being those that acted with no visible, interruptible human checkpoint. This feature reads; it never writes to the day's data.

1.8 Cost and latency

Caching the field/tool definitions cut the governed-API agent's latency **22 % (24.8 s → 19.3 s)** and input cost **64 %**. Our tool catalogue is static, so it is a constant string in the system prompt — same win, no work.

1.9 What this means for SSA — the seven rules

1. Typed tools over the existing semantic layer. No SQL from the model, ever.
2. The model picks and fills; TypeScript computes; a rule draws.
3. Every timestamp is resolved by code, in venue-local time, never by the model.
4. The resolved query is shown back in words and is editable.
5. Every number in the prose is checked against the tool output; unsupported sentences are dropped, exactly as `validateSections()` already does for headlines.
6. Every comparison carries **n**. No effect without its sample size.
7. The answer carries its evidence: photo, clip, scan, tag.

2. Design

2.1 Shape

```

Analysis tab
└─ Ask bar — "Were we quicker on port upwind in the breeze?" [Ask]
      suggested questions as chips when empty
      |
      ▼ POST /api/ai/ask { teamId, boatId, date, question, scope }
      |
      | 1. context pack (deterministic)
      |   what this day IS: venue, boat, phases, sails, TWS range,
      |   races, media counts, which other days exist
      | 2. mistral-medium-3.5-128b + tool catalogue (temperature 0)
      | 3. execute tool calls server-side, RLS-scoped, ≤3 rounds
      | 4. final answer as strict JSON
      | 5. number check → drop unsupported sentences
      |
      |
      ▼ Answer modal
question · interpretation chips · written answer · bottom line
chart(s) drawn from the tool result · the table behind each
evidence: photos / clips / sail scans / tags
"How this was answered": tool, arguments, n, model, EU notice
👍 👎 · follow-up box
  
```

2.2 The five tools

Every argument is enum-constrained; anything the model invents is coerced or the call is rejected with a message it can read and retry.

Tool	Answers	Built on	Chart
<code>compare_phases</code>	"faster on port or starboard?", "which jib in 18 kn?", "how does this day compare with the season?"	<code>expandPhases</code> → filter → <code>groupPhases</code>	grouped bar; line when grouped by an ordered key (TWS band, date)
<code>day_timeries</code>	"when did the breeze go left?", "show heel through race 2"	cloud <code>log_data.rows</code> , decimated	line
<code>list_manoeuvres</code>	"which tacks cost us most?"	stored <code>manoeuvres</code>	bar of distance lost
<code>find_media</code>	"show me the jib at the time", "any scan in that wind?"	<code>photos.analysis_data.inst</code> , <code>videos.start_utc</code> , <code>sail_scans</code> , <code>ssa_tag_events</code>	thumbnail strip
<code>search_notes</code>	"what did we say about the A2?"	<code>ssa_day_notes</code> , tag notes, debriefs	—

`compare_phases` carries the filters — `twsMin/Max`, `twaMin/Max`, `mode`, `tack`, `race`, `sailCombo`, `dateFrom/To` — and a `minPhases` floor so a one-phase band is never called best or worst (the mistake 11 Sep made; `MIN_BAND_PHASES = 3` already encodes it).

Grouping by `date` over a date range *is* the season trend: one tool, not two.

2.3 Why server-side, and what it costs

The route reads the **cloud**, not the browser's IndexedDB — the rule from `CLAUDE.md`: anything derived on the importing machine is blank for everyone else. The price is that a day must have its phase stats stored (`session_phase_stats`, written when someone opens Performance charts). When the row is missing, the tool says so in words the model relays, the way `.../headlines` already returns 409.

2.4 What it will refuse

No answer beats a wrong one. It declines and offers alternatives when: the day has no stored stats; a filter leaves fewer than `minPhases` phases; the question needs a channel the boat does not log; or the question is about something outside the five tools. It never falls back to guessing from memory.

3. Build plan

#	Step	Files	Done when
1	Scaleway client with tool-calling	<code>lib/ai/scaleway.ts</code>	one <code>chat()</code> , EU-only, abortable, reuses <code>extractJson</code>
2	Tool catalogue + arg validation	<code>lib/ai/askTools.ts</code>	invalid enum → readable error, not a crash; chips render
3	Deterministic charts	<code>lib/ai/askCharts.ts</code>	spec from table shape; no model input
4	Number check	<code>lib/ai/askVerify.ts</code>	reuses <code>numbersIn</code> / <code>factNumbers</code> ; unsupported sentence dropped
5	Agent loop	<code>lib/ai/askRun.ts</code>	pure; executor injected; ≤3 rounds; testable with a fake model
6	Data executors	<code>lib/ai/askData.ts</code>	RLS-scoped reads for phases, log, media, notes
7	Route	<code>api/ai/ask/route.ts</code>	auth, <code>canUseAI</code> , 60 s cap, logs to <code>ai_query_log</code>

#	Step	Files	Done when
8	Log table	<code>0087_ai_query_log.sql</code>	one log for every AI surface; read follows the day
9	Ask bar	<code>components/analytics/AskPanel.jsx</code>	suggestions when empty; disabled with a reason, never silently
10	Answer modal	<code>components/analytics/AskAnswerModal.jsx</code>	chips, answer, charts, tables, evidence, provenance, feedback, follow-up
11	Chart renderer	<code>components/analytics/AskChart.jsx</code>	bar/line in SSA's chart style
12	Tests	<code>__tests__/</code>	tools, verify, charts, loop

Migration number **0087** — `main` is at 0086. That sidesteps the `0055 numbering` collision on `ai-sovereign-mistral`, but not the real one: `ai_query_log` **already exists on the remote database**, created by that branch's 0055 even though the migration was never recorded as applied. The first push failed on it — *relation already exists*, then *column session_date does not exist*.

So 0087 **adopts and extends** the table instead of creating one. It keeps 0055's vocabulary where the two overlap (`rating / correction / rated_by / rated_at`, `latency_ms`) and adds `session_date`, `steps`, `risk`, `used_tools`; `route` says which surface wrote the row. One log for every AI surface is what the roadmap asks for, and two names for one thing is the mess it avoids. Both paths — a fresh database and one already carrying the 0055 table — converge on the same shape, and the branch's `CREATE TABLE IF NOT EXISTS` no-ops if it ever merges.

One deliberate narrowing: 0055 let any coach or team manager read the whole team's log. 0087 makes reading follow the **day**, as every other derived table in this repo does — a question quotes its day's numbers back, so somebody who cannot see that day should not read it. You always see your own questions regardless. Curating few-shot examples across a team is a service-key job in a script, not something that needs to be reachable from a browser.

Deliberately not in this slice

pgvector/embeddings (keyword search first — it answers "what did we say about the A2" today); the read-only SQL escape hatch; debrief observations as rows; a golden eval set for these questions. Each is in the roadmap and each wants this surface to exist first, so the eval can be built from what people really ask.